Компьютер Загварчлал, Симуляц
(Бие даалт)

Багш Д. Батмөнх

2026 оны 3-р сарын 19

Бие даалт №1: Шүхэртэй үсрэлтийг загварчлах

Бодлогын үндсэн тавилт

Хугацаа: 03/18–04/15

Аврагч 4000 метрийн өндрөөс шүхэртэй үсэрч, 60 секундийн хугацаанд чөлөөт уналтын дараа шүхрээ задална. Аврагчийн биеийн масс $m = 85$ кг бөгөөд шүхэргүй үеийн эсэргүүцлийн тогтмол нь $C = 0.7$, буулгалтын талбай нь $A = 0.5$ м^2 болно. Харин шүхэртэй үеийн эсэргүүцлийн тогтмол нь $C = 1.5$, буулгалтын талбай нь $A = 30$ м^2. Далайн төвшний агаарын нягт $D_0 = 1.225$ кг/м^3, далайн төвшнөөс дээш $D(h) = D_0 \cdot e^{-h/8500}$ бөгөөд хүндийн хүчний хурдатгал $g = 9.81$ м/с^2.

Ньютоны 2-р хууль:

$$m\frac{dv}{dt} = mg - \tfrac{1}{2}CD(h)Av^2$$

Ньютоны үрэлт:

$$\frac{dh}{dt} = -v$$

Өндрөөс хамаарсан агаарын нягт:

$$D(h) = D_0 \cdot e^{-h/8500}$$

энд, h нь газраас дээш өндөр, v нь доош чиглэсэн хурд болно. Тэгвэл дээрх томьёонуудыг ашиглан дараах даалгавруудыг гүйцэтгэнэ:

a) Суурь загварыг хоёр үе шаттай загварчил ($\Delta t = 0.1$ с):

- Эхний үе (0–60 секунд хүртэл): Чөлөөт буюу шүхэргүй уналт ($C = 0.7$, $A = 0.5$)
- Удаах үе (60 секундээс газардах хүртэл): Шүхэртэй уналт ($C = 1.5$, $A = 30$)
- Шүхэр задрах агшинд параметрүүд нэг дор хэрхэн өөрчлөгдөхийг тооцоолж зураглана.

b) Агаарын нягтын шинжилгээ:

- Өндрөөс хамаарсан $D(h) = D_0 \cdot e^{-h/8500}$ агаарын нягтыг $\Delta t = 0.1$ секундийн тасралттайгаар тооцоол.
- Загварчлалыг тогтмол нягттай ($D = 1.2$) үеийн загварчлалтай харьцуулж, зурагла.

c) Тогтмол хурд (terminal velocity): $v_t = \sqrt{2mg/(CDA)}$

- Шүхэргүй болон шүхэртэй үеийн тогтмол хурднуудыг дээрх томьёогоор тооцоол.
- Үлгэрчлэлийн үр дүнтэй харьцуулж баталгаажуул.

d) Аюулгүй байдлын шинжилгээ:

- Газарт буух үеийн хурд 6 м/с-ээс бага байх нөхцөлийг хангах эсэхийг шалга.
- Хэрэв хангагдахгүй бол шүхрийн A утга ямар үед хангагдахыг ол.
- Шүхэр 50, 60, эсвэл 70 секундэд задарвал буух хурд хэрхэн өөрчлөгдөхийг зурагла.

e) Масс болон өндрийн шинжилгээ:

- Биеийн масс нь $60, 70, 80, 90, 100$ кг байх үеийн буух хурдыг харьцуулах зурагла.
- Үсрэлтийн өндөр $2000, 3000, 4000, 5000$ м байх үеийн ялгааг шинжил.
- Ингэхдээ colormap ашиглан өнгөөр ялга

f) Рунге-Куттын 4-р эрэмбийн арга (RK4):

- Эйлерийн аргын оронд RK4 аргыг ашиглаж нарийвчлалын ялгааг харьцуул.
- Δt нь 1.0, 0.1, 0.01 секунд байх үед хоёр аргын зөрүүг хүснэгт, зураглалаар үзүүл.

Гүйцэтгэх ажлын бүтэц

Дараах 6 алхмын дагуу загварчилж тайлан бичнэ:

1. Асуудлыг шинжлэх: Асуудлын мөн чанар, зорилго, ангилал (тодорхой/санамсаргүй, хөдөлцтэй/хөдөлцгүй, тасралттай/тасралтгүй)

2. Загварчлалыг томьёолох: Хувьсагч, нэгж, таамаглал, тэгшитгэл. Хялбаршуулах шалтгааныг тайлбарлах

3. Загварчлалыг хэрэгжүүлэх: Python код (функц, давталт, NumPy).

4. Баталгаажуулах: Yp дүнг онолын шийд, бодит өгөгдөл, эсвэл хязгаарын нөхцөлөөр шалгах. Шалгах болон баталгаажуулах хоёрын ялгааг тодруулах

5. Тайлан боловсруулах: Зураглал (matplotlib), хүснэгт, дүн шинжилгээний тайлбар. Axis labels, title, grid, legend бүрэн байх

6. Сайжруулах: Өгөгдөхүүн өөрчлөх, нэмэлт нөхцөл нэмэх, хязгаарлалтыг тодорхойлох. Давтагдсан кодыг сайжруулах (refactor): тусгай тохиолдлуудыг нэг ерөнхий функц болгон нэгтгэх.

Төсөл илгээх

`https://gitlab.com/bataak/f.csa304-2526` сангийн `prj1/` хавтаст буй `README.md` файлд заасан бүтцийн дагуу бие даалтын ажлыг гүйцэтгэнэ.

Кодын шаардлага

1. Кодын бүтэц: Код нь дараах гурван хэсгээс бүрдэнэ: `simulation.py`, `analysis.py` болон `visualization.py`. Jupyter notebook (`.ipynb`) ашиглахгүй.

2. Функцийн загвар: Загварчлалын үндсэн функцүүд нь параметр хүлээн авч, yp дүнг буцааж, дуудагдах боломжтой байна. Жишээ:

```
# src/simulation.py
import numpy as np

def simulate_logistic(P0, r, M, years, dt=1.0):
    """
    Логистик өсөлтийн үлгэрчлэл.

    Parameters:
        P0 : float   -- анхны тоо толгой
        r  : float   -- өсөлтийн хурд
        M  : float   -- бэлчээрийн даац
        years : int  -- үлгэрчлэх хугацаа (жилээр)
        dt : float   -- хугацааны алхам

    Returns:
```

```
        t : np.ndarray -- хугацааны массив
        P : np.ndarray -- тоо толгойн массив
    """
    n = int(years / dt)
    t = np.linspace(0, years, n + 1)
    P = np.zeros(n + 1)
    P[0] = P0
    for i in range(n):
        P[i+1] = P[i] + r * (1 - P[i]/M) * P[i] * dt
    return t, P
```

3. Бичил шалгалт: `unittest` эсвэл `pytest` ашиглан бичил шалгалт бичнэ. Хамгийн багадаа 5 шалгалт бичнэ:

```
# tests/test_simulation.py
import pytest
import numpy as np
from src.simulation import simulate_logistic

def test_initial_value():
    """Анхны утга зөв эсэх."""
    t, P = simulate_logistic(500, 0.3, 2000, 20)
    assert P[0] == 500

def test_equilibrium():
    """P0 = M үед өөрчлөлт байхгүй."""
    t, P = simulate_logistic(2000, 0.3, 2000, 10)
    np.testing.assert_allclose(P[-1], 2000, atol=1)

def test_growth_direction():
    """P0 < M үед тоо толгой өсөх ёстой."""
    t, P = simulate_logistic(100, 0.3, 2000, 5)
    assert P[-1] > P[0]

def test_carrying_capacity():
    """Урт хугацаанд M-д дөхөх ёстой."""
    t, P = simulate_logistic(100, 0.3, 2000, 100)
    np.testing.assert_allclose(P[-1], 2000, atol=10)

def test_negative_input():
    """Сөрөг анхны утга зохицуулах."""
    with pytest.raises(ValueError):
        simulate_logistic(-100, 0.3, 2000, 20)
```

4. Ажиллуулах:

```
cd students/B23XXXXX/
pip install -r requirements.txt
pytest tests/ -v
python src/simulation.py
```

5. Кодыг сайжруулах (refactor): Даалгаврыг 3 үе шаттайгаар гүйцэтгэж, үе шат бүрийн дараа `pytest tests/ -v` шалгахад бүгд алдаагүй байх ёстой.

   1-р шат. Эхлээд сэдвийнхээ нэг тохиолдлыг `simulation.py` дотор бүгдийг нэг функцэд бичнэ. Жишээлбэл бэлчээрийн даацыг загварчлах бодлогын хувьд:

```
# simulation.py -- 1-р үе шат
def simulate_sheep(P0, r, M, years):
    """Зудгүй, энгийн загвар."""
    P = [P0]
    for _ in range(years):
        P.append(P[-1] + r*(1-P[-1]/M)*P[-1])
    return P
```

   2-р шат. Зудын нөхцөл, бэлчээрийн даацын өөрчлөлт зэргийг нэмэхэд код давтагдаж эхэлнэ. Тэр үед дараах сайжруулалтыг (refactor) хийнэ:

   - Change Signature: Функцийн параметрийн бүтцийг өөрчлөх буюу `events`, `dt` параметр нэмэх. Дуудагдсан тохиолдол (test, visualization) бүрийг шинэчлэгдэх ёстой.
   - Extract Method: Нэг том функц доторх бүлэг логикийг тусад нь функц болгон задлах, жишээ нь `apply_event(P, impact)` функц.
   - Move Function: `find_equilibrium()`, `growth_ratio()`) зэрэг шинжилгээний функцүүдийг `simulation.py` файлаас `analysis.py` файл руу зөөх. Зөөсний дараа `simulation.py`, `test_simulation.py`, `visualization.py` дахь бүх `import`-ууд автоматаар шинэчлэгдэх ёстой. Зөөх үйлдлийг IDE refactor функц ашиглан хийнэ.

   3-р шат. Зураглалын кодыг `visualization.py` руу зөөх. Энэ модуль нь `simulation.py` болон `analysis.py` хоёроос `import` хийдэг учир гурван модулийн хоорондын хамааралт бүтэц үүсэх ба ийм бүтэцтэй `simulate_logistic()` функцийн нэр эсвэл параметрийг өөрчилбөл эдгээр 3 файлд нэг зэрэг өөрчлөлт хийх шаардлагатай болно.

Баримт бичгийн шаардлага

- Тайланг `template.ott` загвар файл ашиглан LibreOffice программаар бэлтгэнэ.
- «Монгол үсгийн дүрэм»-ийг баримтална (`https://zuv.bichig.dev`).
- Зураглал бүрд тэнхлэгийн нэр, нэгж, гарчиг, торлог бүрэн байна.
- Багаар (2–3 хүн) гүйцэтгэж болно. Шалгуулахдаа тус тусдаа шалгуулна.

Бие даалт №2: Улаанбаатар хотын замын уулзварын түгжрэлийг загварчлах

Бодлогын үндсэн тавилт

Хугацаа: 04/15–05/07

Стохастик системийг Монте-Карло аргаар загварчилж, санамсаргүй хувьсагчтай ажиллах, статистик шинжилгээ хийх чадварыг эзэмшинэ. Загварчлах үйлдлийн 6 үе шатыг гүйцэтгэнэ.

Улаанбаатар хотын уулзвар бүр 4 чиглэлтэй бөгөөд чиглэл бүрээс минутад дунджаар λ машин ирдэг. Гэрлэн дохионы солигдох мөчлөг 90 секунд (ногоон гэрэл: 40 секунд, улаан гэрэл: 50 секундийн хугацаанд асдаг). Ногоон гэрлээр 1 секундэд 1 машин нэвтрэх боломжтой. Замын ачаалал 08:00–09:00 болон 17:00–19:00 цагийн хооронд өндөр байдаг. Оргил цаг болон оргил бус цагийн нэвтрүүлэх чадамж λ утгаар ялгаатай.

Монте-Карло аргыг ашиглан стохастик үлгэрчлэл зохиоход шаардлагатай шалгуур үзүүлэлтүүд:

- Машины ирэх хугацааны завсар нь $\mathrm{Exp}(1/\lambda)$ илтгэгч тархалттай,
- Дараалын урт нь зогсож буй машины тоотой тэнцүү,
- Хүлээх хугацаа нь нэг машины дундаж хүлээлтийн хугацаатай тэнцүү,
- Оргил цагийн λ утга нь суурь λ утгаас 2–3 дахин их

Пуассоны тархалтыг ашиглан дараах нөхцөлүүдийг харгалзан загварчилна:

a) Нэг чиглэлт суурь загварчлал:

   - Нэг чиглэлийн хувьд 1 цагийн үлгэрчлэл зохио ($\lambda = 15$ машин/мин)

- Секунд бүрд шинэ машин ирэх эсэхийг Пуассоны тархалтаар шийдэж, гэрлэн дохиогоор нэвтрүүлэх, дараалын уртыг бүрт-гэх
- 1000 удаагийн давталтаар, дундаж дараалын урт, дундаж хүлээлтийг тооцоол.

b) 4 чиглэлт загвар:

- Зүүнээс баруун тийш (ногоон 40 секунд асна) болон хойноос өмнө тийш (ногоон 50 секунд асна) хуваарилалт
- 4 чиглэл зэрэг буюу чиглэл бүрийн дараалал тусдаа байх
- Бүх чиглэлийн нийт хүлээлтийг нэгтгэж дундажла

c) Хоногийн хэлбэлзэл:

- 24 цагийн $\lambda(t)$ нэвтрүүлэх чадамжийг боловсруул: оргил цагт (08–09, 17–19) $\lambda = 25$, өдрийн цагт (09–17) $\lambda = 15$, шөнийн цагт (22–06) $\lambda = 3$, бусад үед $\lambda = 10$
- 24 цагийн үлгэрчлэл зохиож, цаг тутмын дундаж дараалын уртыг зурагла

d) Гэрлэн дохионы оновчлол:

- Ногоон/улаан гэрлийн асах хугацааны хуваарилалтыг 30/60, 40/50, 50/40, 60/30 болгон туршиж, аль хуваарилалт хамгийн бага хүлээлттэй болохыг олох
- Оргил цагт болон оргил бус цагт ялгаатай мөчлөг ашиглах загварыг санал болгох

e) Статистик шинжилгээ:

- Хүлээх хугацааны хистограмм зурж, тархалтын хэлбэрийг тайл-барлах
- Давталтын тоог $N = 100, 500, 1000, 5000, 10000$ болгох үеийн дундаж утгын тогтворжилтыг зураглах
- 95% итгэлцлийн завсрыг тооцоол: $\bar{x} \pm 1.96 \cdot s/\sqrt{N}$

f) Бодит өгөгдөлтэй харьцуулах:

- УБ хотын нэг уулзварын бодит ажиглалтын өгөгдлийг цуг-луул (жишээ нь, 15 минут тоолох, эсвэл Google Maps-аас оргил цагийн мэдээлэл авах)
- Загварын үр дүнг бодит өгөгдөлтэй харьцуулж, загварын хяз-гаарлалтыг тодорхойл

Гүйцэтгэх ажлын бүтэц

Өмнөх төслийн гүйцэтгэх ажлын бүтцийг харна уу.

Төслийн бүтэц

`https://gitlab.com/bataak/f.csa304-2526` сангийн `prj2/` хавтсанд дараах бүтцээр оруулна:

Кодын шаардлага

1. **Модулийн бүтэц:** Загварын тэгшитгэлийг `model.py`, давталтын логикийг `simulation.py`, шинжилгээг `analysis.py` болон зураглалыг `visualization.py` гэж тусгаарлана. Jupyter notebook (`.ipynb`) ашиглахгүй.

2. **Функцийн загвар:** Жишээ:

```
# src/model.py -- Замын загварын тэгшитгэлүүд
import numpy as np

def generate_arrivals(lam, duration_sec, rng=None):
    """
    Пуассоны тархалтаар машины ирэлтийг үүсгэнэ.

    Parameters:
        lam : float -- минутад ирэх машины дундаж тоо
        duration_sec : int -- хугацаа (секундээр)
        rng : np.random.Generator -- seed удирдах

    Returns:
        arrivals : np.ndarray -- секунд бүрд ирсэн машины тоо
    """
    if rng is None:
        rng = np.random.default_rng()
    lam_per_sec = lam / 60.0
    arrivals = rng.poisson(lam_per_sec, size=duration_sec)
    return arrivals

def simulate_intersection(lam, green_sec, red_sec,
                          duration_sec, rng=None):
    """
    Нэг чиглэлийн уулзварын симуляц.
```

```
    Returns:
        queue_log : np.ndarray -- секунд бүрийн дараалын урт
        wait_times : list -- машин бүрийн хүлээсэн хугацаа
    """
    arrivals = generate_arrivals(lam, duration_sec, rng)
    cycle = green_sec + red_sec
    queue = 0
    queue_log = np.zeros(duration_sec)
    wait_times = []

    for t in range(duration_sec):
        queue += arrivals[t]
        phase = t % cycle
        if phase < green_sec and queue > 0:
            wait_times.append(queue)  # ойролцоо хүлээлт
            queue -= 1
        queue_log[t] = queue

    return queue_log, wait_times
```

3. **Бичил шалгалт (unit test):** `pytest` ашиглан хамгийн багадаа **7 шалгалт** бичнэ (Бие даалт 1-ээс илүү). Стохастик шалгалтад **seed тогтоох** шаардлагатай:

```
# tests/test_model.py
import pytest
import numpy as np
from src.model import generate_arrivals, simulate_intersection

@pytest.fixture
def fixed_rng():
    """Давтагдах үр дүн гаргах seed."""
    return np.random.default_rng(42)

def test_arrivals_shape(fixed_rng):
    """Гаралтын урт duration-тай тэнцүү."""
    arr = generate_arrivals(15, 3600, fixed_rng)
    assert len(arr) == 3600

def test_arrivals_mean(fixed_rng):
    """Дундаж нь lambda/60-д ойрхон."""
    arr = generate_arrivals(15, 36000, fixed_rng)
    assert abs(arr.mean() - 15/60) < 0.02
```

```
def test_zero_lambda():
    """lambda=0 бол машин ирэхгүй."""
    rng = np.random.default_rng(0)
    arr = generate_arrivals(0, 3600, rng)
    assert np.all(arr == 0)

def test_queue_never_negative(fixed_rng):
    """Дараалал хэзээ ч сөрөг болохгүй."""
    q, _ = simulate_intersection(15, 40, 50, 3600,
                                 fixed_rng)
    assert np.all(q >= 0)

def test_longer_green_less_queue(fixed_rng):
    """Ногоон урт бол дараалал богино."""
    rng1 = np.random.default_rng(42)
    rng2 = np.random.default_rng(42)
    q_short, _ = simulate_intersection(15, 30, 60, 3600,
                                       rng1)
    q_long, _ = simulate_intersection(15, 60, 30, 3600,
                                      rng2)
    assert q_long.mean() < q_short.mean()

def test_high_lambda_longer_queue():
    """Lambda ихсэхэд дараалал уртасна."""
    rng1 = np.random.default_rng(42)
    rng2 = np.random.default_rng(42)
    q_low, _ = simulate_intersection(10, 40, 50, 3600,
                                     rng1)
    q_high, _ = simulate_intersection(25, 40, 50, 3600,
                                      rng2)
    assert q_high.mean() > q_low.mean()

def test_reproducibility():
    """Ижил seed = ижил үр дүн."""
    rng1 = np.random.default_rng(123)
    rng2 = np.random.default_rng(123)
    q1, _ = simulate_intersection(15, 40, 50, 3600, rng1)
    q2, _ = simulate_intersection(15, 40, 50, 3600, rng2)
    np.testing.assert_array_equal(q1, q2)
```

4. **Ажиллуулах:**

```
cd students/B23XXXXX/
```

```
pip install -r requirements.txt
pytest tests/ -v
python src/simulation.py
```

5. **Кодыг сайжруулах (refactor):** Бие даалт 2-т refactor нь бие даалт 1-ээс илүү нарийн — олон модуль хоорондын хамаарлыг зөв удирдана. 3 үе шаттайгаар гүйцэтгэнэ. Үе шат бүрийн дараа `pytest tests/ -v` бүгд ногоон байх ёстой.

   Үе шат 1. Бүх кодыг эхлээд `simulation.py` дотор бичнэ (машины ирэлт үүсгэх, дараалал удирдах, статистик тооцоо, зураглал — бүгд нэг файлд).

   Үе шат 2. Нэг файлаас 4 модуль уруу задлах:

```
# ӨМНӨ: simulation.py дотор бүгд холилдсон
def generate_arrivals(lam, duration, rng): ...
def simulate_intersection(lam, green, red, ...): ...
def simulate_full_day(lam_profile, ...): ...
def calc_mean_wait(wait_times): ...
def calc_confidence_interval(data, conf): ...
def plot_queue(queue_log): ...
def plot_histogram(wait_times): ...

# ДАРАА: 4 модуль уруу задалсан
model.py         → generate_arrivals(),
                   simulate_intersection()
simulation.py    → simulate_full_day(),
                   run_monte_carlo(n_trials, ...)
                   (model.py-аас import хийнэ)
analysis.py      → calc_mean_wait(),
                   calc_confidence_interval(),
                   find_optimal_signal()
                   (simulation.py-аас import хийнэ)
visualization.py → plot_queue(), plot_histogram(),
                   plot_convergence()
                   (simulation + analysis-аас import)
```

   Энэ задралд дараах refactor үйлдлүүд шаардлагатай:

   - **Move Function (F6):** Функцийг нэг файлаас нөгөө уруу зөөх. Зөөсний дараа хуучин файл дахь `import`, бүх дуудсан газар шинэчлэгдэх ёстой.
   - **Change Signature:** Жишээ нь, `simulate_intersection()` функцэд `capacity_per_sec` параметр нэмэхэд (секундэд 1 биш 2

машин нэвтрэх тохиолдол) бүх test, analysis, visualization дахь дуудалтууд хамт шинэчлэгдэх ёстой.

- **Extract Method:** Жишээлбэл, `simulate_full_day()` дотроос `get_lambda(hour)` гэсэн цагаас хамаарсан λ олох функц гаргах.

Ye шат 3. 4 модулийн хоорондын хамаарлын бүтэц бүрэн үүснэ. `generate_arrivals()` функцийн параметрийг өөрчилбөл 4 файл дахь бүх import, дуудалт, test нэг дор зөв шинэчлэгдэх шаардлагатай.

Баримт бичгийн шаардлага

- Тайланг `template.ott` загвар файл ашиглан LibreOffice программаар бэлтгэнэ.
- «Монгол үсгийн дүрэм»-ийг баримтална (`https://zuv.bichig.dev`).
- Зураглал бүрд тэнхлэгийн нэр, нэгж, гарчиг, торлог бүрэн байна.
- Багаар (2–3 хүн) гүйцэтгэж болно. Шалгуулахдаа тус тусдаа шалгуулна.